

Data Engineering Interview Guide

Top 18 Fundamentals: Code Solutions with Explicit Inputs, Outputs & Hash Maps

1. SQL QUESTIONS (1–6)

1. Find top 3 highest-earning employees per department

```
WITH Ranked AS (  
  SELECT emp_id, emp_name, department, salary,  
         DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) as rnk  
  FROM employees  
) SELECT emp_id, emp_name, department, salary FROM Ranked WHERE rnk <= 3;
```

INPUT (EMPLOYEES TABLE):

emp_id	dept	salary
101	Tech	120000
102	Tech	110000
103	Tech	105000
104	Tech	90000

OUTPUT RESULT:

emp_id	dept	salary
101	Tech	120000
102	Tech	110000
103	Tech	105000

2. Identify employees hired in the past 6 months

```
SELECT emp_id, emp_name, hire_date FROM employees  
WHERE hire_date >= CURRENT_DATE - INTERVAL '6 month';
```

INPUT DATA (CURRENT DATE: 2026-03-31):

emp_id	emp_name	hire_date
101	Alice	2025-11-15
102	Bob	2025-05-10

OUTPUT RESULT:

emp_id	emp_name	hire_date
101	Alice	2025-11-15

3. Compute cumulative salary totals grouped by employee

```
SELECT emp_id, pay_month, salary,  
       SUM(salary) OVER (PARTITION BY emp_id ORDER BY pay_month) AS cumulative_total  
FROM salary_history;
```

INPUT (SALARY_HISTORY):

emp_id	month	salary
101	2026-01	5000
101	2026-02	5500

OUTPUT RESULT:

emp_id	month	salary	cumulative_total
101	2026-01	5000	5000
101	2026-02	5500	10500

4. Count which departments have more than 5 employees

```
SELECT department, COUNT(emp_id) AS total_employees  
FROM employees GROUP BY department HAVING COUNT(emp_id) > 5;
```

Input: Departments with 7 (Tech), 3 (HR), 6 (Sales) staff.

Output: Tech: 7, Sales: 6

5. Eliminate duplicate entries without using DISTINCT keyword

```
SELECT customer_id, email FROM (  
  SELECT customer_id, email,  
         ROW_NUMBER() OVER (PARTITION BY email ORDER BY customer_id) as row_num  
  FROM customers  
) WHERE row_num = 1;
```

Input: [(1, 'a@x.com'), (2, 'a@x.com'), (3, 'b@x.com')]

Output: [(1, 'a@x.com'), (3, 'b@x.com')]

6. Identify employees making more than their department's average

```
WITH DeptAvg AS (  
    SELECT department, AVG(salary) AS avg_sal FROM employees GROUP BY department  
)  
SELECT e.emp_id, e.emp_name, e.department, e.salary, d.avg_sal  
FROM employees e JOIN DeptAvg d ON e.department = d.department  
WHERE e.salary > d.avg_sal;
```

Input: Dept Avg = \$100k; Employees: Alice (\$120k), Bob (\$80k).

Output: Alice (\$120k > \$100k)

2. PYTHON QUESTIONS (7-12)

7. Anagram Check using Hash Map Algorithm (Explicit Counter)

```
def is_anagram(s1: str, s2: str) -> bool:  
    s1, s2 = s1.lower().replace(" ", ""), s2.lower().replace(" ", "")  
    if len(s1) != len(s2): return False  
  
    hash_map = {}  
    for char in s1:  
        if char in hash_map:  
            hash_map[char] += 1  
        else:  
            hash_map[char] = 1  
  
    for char in s2:  
        if char not in hash_map or hash_map[char] == 0:  
            return False  
        hash_map[char] -= 1  
  
    return True
```

Input 1: s1 = "listen", s2 = "silent" | **Output 1:** True

Input 2: s1 = "hello", s2 = "world" | **Output 2:** False

8. Reverse a string using only loops (no slicing)

```
def reverse_string(s: str) -> str:  
    reversed_s = ""  
    for char in s:  
        reversed_s = char + reversed_s  
    return reversed_s
```

Input: "DataEngine" | **Output:** "enignEataD"

9. First unique character using Hash Map Count Check

```
def first_unique_char(s: str) -> int:  
    counts = {}  
    for char in s:  
        if char in counts:  
            counts[char] += 1  
        else:  
            counts[char] = 1  
  
    for idx, char in enumerate(s):  
        if counts[char] == 1:  
            return idx  
    return -1
```

Input: s = "leetcode" | **Hash Map Built:** {'l':1, 'e':3, 't':1, 'c':1, 'o':1, 'd':1}

Output Index: 0 (Character: 'l')

10. Convert a nested list into a single flat list

12. Find duplicates in a list using minimal memory (Hash Map / Set)

```
def find_duplicates(nums):
    counts = {}
    duplicates = []
    for num in nums:
        if num in counts:
            counts[num] += 1
        else:
            counts[num] = 1

    for num, count in counts.items():
        if count > 1:
            duplicates.append(num)
    return duplicates
```

Input List: [1, 3, 4, 2, 2, 5, 3, 6] | **Output Duplicates:** [2, 3]

3. PYSPARK QUESTIONS (13–18)

13. Define narrow and wide transformations

Narrow Transformations: Operations where each input partition contributes to at most one output partition (e.g., `map()`, `filter()`, `withColumn()`). No network shuffle required.

Wide Transformations: Operations that require data from multiple partitions to be combined across nodes, triggering a network Shuffle (e.g., `groupBy()`, `join()`).

Input DataFrame: 3 partition execution blocks.

Output (Narrow vs Wide): `filter()` maintains partition boundaries; `groupBy()` shuffles data across nodes.

14. Explain how Spark partitions data and why it matters

Spark splits large datasets into partitions to process them concurrently across worker nodes. Proper partitioning optimizes CPU parallelism, avoids data skew bottlenecks, and speeds up reads via partition pruning.

Input Dataset: 100 GB unpartitioned CSV log file.

Output Strategy: Partitioning by year/month splits data into balanced sub-directories, allowing targeted reads.

15. Efficiently process DataFrames with missing values

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()

data = [("Alice", 100), ("Bob", None), (None, 80)]
df = spark.createDataFrame(data, ["name", "score"])

# 1. Drop nulls in key column
df_clean = df.dropna(subset=["name"])

# 2. Fill remaining nulls with default values
df_filled = df.fillna({"score": 0, "name": "Unknown"})
df_filled.show()
```

INPUT PYSPARK DATAFRAME:

name	score
Alice	100
Bob	null
null	80

OUTPUT PYSPARK DATAFRAME:

name	score
Alice	100
Bob	0
Unknown	80

16. Describe window functions and appropriate use cases

Window functions perform evaluations across related row groups (frames) without collapsing rows into a single aggregate group.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, col

spec = Window.partitionBy("dept").orderBy(col("salary").desc())
df_ranked = df.withColumn("rank", row_number().over(spec))
```

INPUT DATAFRAME:

dept	salary
Sales	9000
Sales	7000

OUTPUT DATAFRAME:

dept	salary	rank
Sales	9000	1
Sales	7000	2

17. Determine whether to use cache or persist

`cache()` stores DataFrames with default `MEMORY_ONLY` storage level. `persist()` allows customizing storage levels (e.g., `MEMORY_AND_DISK`) to spill extra data to disk safely.

Scenario A: Medium DataFrame referenced repeatedly in loop -> **Use:** `df.cache()`

Scenario B: Huge DataFrame exceeding RAM limits -> **Use:** `df.persist(StorageLevel.MEMORY_AND_DISK)`

18. Choose between repartition and coalesce functions

```
# Scenario 1: Rebalance data across cluster nodes
df_balanced = df.repartition(10)

# Scenario 2: Combine existing partitions to write fewer output files
df_single = df.coalesce(1)
```

INPUT & ACTION:

- Dataset split across 20 skewed partitions
- Operation: `coalesce(2)`

OUTPUT RESULT:

- Combines partitions locally without network shuffle
- Produces exactly 2 files on disk

Summary Checklist for Candidates:

- ✓ **SQL:** Window functions (`DENSE_RANK`, `SUM OVER`) eliminate complex joins.
- ✓ **Python:** Explicit Hash Maps (if `key in map: map[key] += 1`) guarantee $O(N)$ frequency counting.
- ✓ **PySpark:** Distinguish narrow vs. wide transformations to minimize costly shuffle steps.